IST 615 – CLOUD MANAGEMENT

KUBERNETES AND MICROSERVICES

Jim Garrisi

Adjunct Professor
School of Information Studies
Syracuse University

Outline

- Announcements
- Recap
- Serverless computing
- Micro-services
- WebAssembly
- Lab 5 description

Announcements

- Preparations for project
  - Goal: Use cloud services (at least two) to complete a “specific” task

    Examples:
    - Build a data analysis pipeline to process dataset X
    - Automatically trigger a data analysis pipeline when a file is placed in cloud storage
    - Send an alert via email when a sensor’s reading exceeds value Y
    - Deploy a database that automatically logs some variable/changing information

Final Project

- You can work in a group – maximum 3 students
- Have a "task" defined for your project
  - Submit via Blackboard a 2 to 4 page proposal detailing team members, project goals and cloud services/tools to be used plus any additional resources needed.
    - The description of cloud services you will use, must focus on how they support your solution, not on the generic description of what the service does as stated by the web/marketing material of the cloud service platform.
  - A diagram that illustrates the architecture of your cloud solution and/or the interaction between services (inputs / outputs) must be included in your proposal.
  - Links / references / sources of information to be used
  - Instructor might suggest that some smaller teams work together
  - Instructor to send "brief" feedback to groups by November 5
  - NOTE: For projects using AWS services… use a PERSONAL account, not an AWS Academy account!
- Complete Final Project by last day of class

Labs – Troubleshooting forum

- A discussion forum to submit issues that you may have while executing a lab has been created in Blackboard

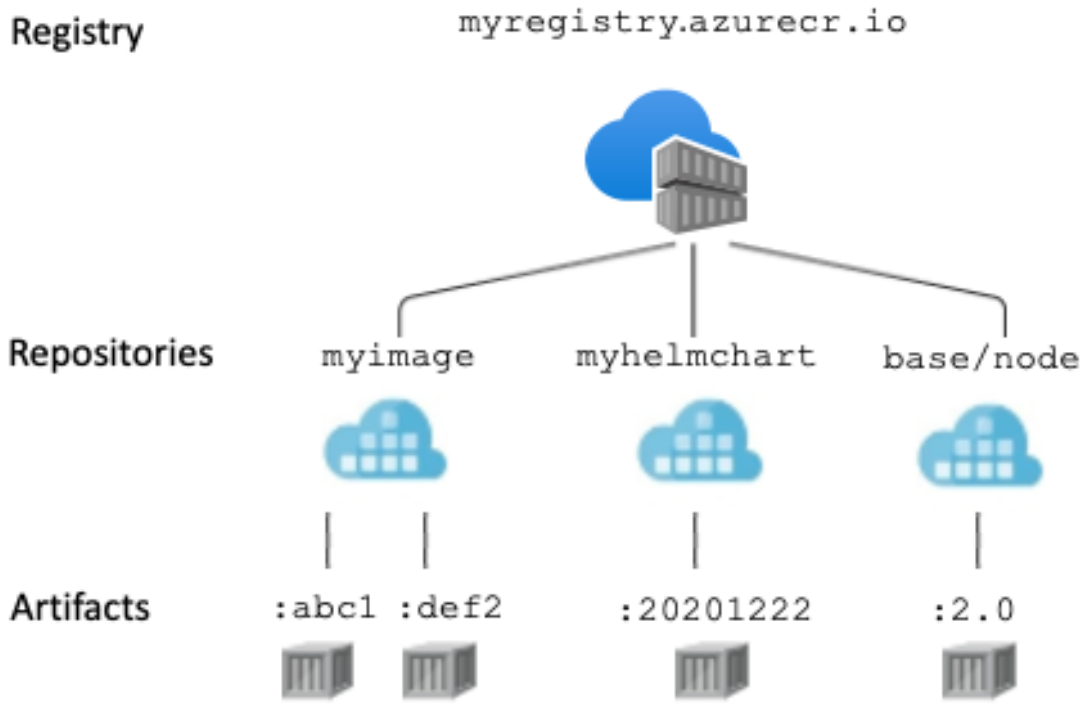
- Please describe your problem clearly and mention at least 2 approaches to solve the problem that you have already tried.
- After posting an issue, e-mail the instructor if you want him to get involved
- Other students can also proceed to provide help and it would be counted as participation in class (see Participation points)
- If possible, include a Kaltura Media video capture of the problem and try not to include private credentials in the video.

Recap

Managing containers – Introduction (1)

- You can push and pull container images to/from a container registry, which is something like a "GitHub" for container images.

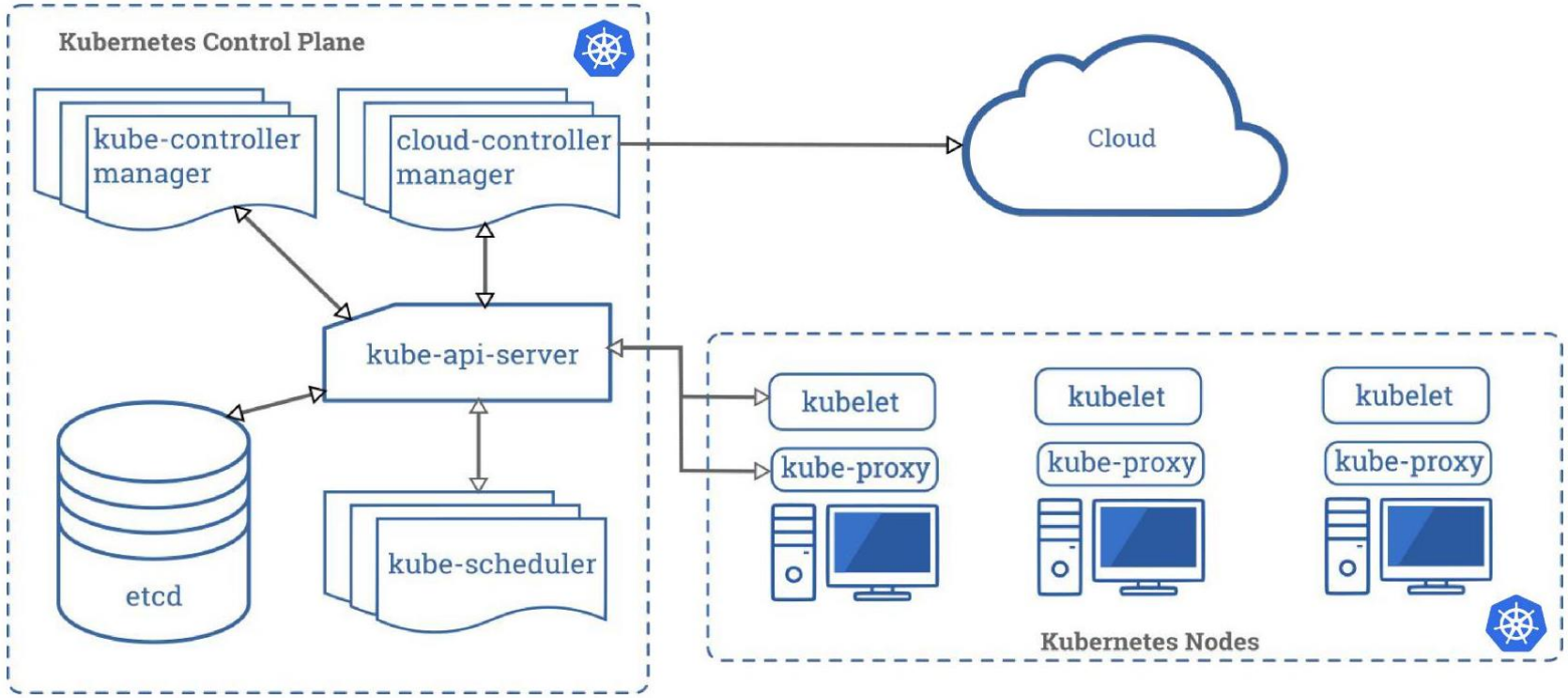
- Registry: service that stores and distributes container images and related artifacts
  - E.g. Docker Hub, Quay.io, etc.
- Repository: Collection of container images or other artifacts (in a registry) that have the same name but different tags
- Artifact: container image, helm charts, manifests, etc.
- Tag: Alpha-numeric string that identifies an artifacts version



Source: https://docs.microsoft.com/en-us/azure/container-registry/container-registry-concepts

Kubernetes

- Kubernetes (also known as k8s) is an open source container orchestration platform that automates many of the manual processes involved in deploying, managing, and scaling containerized applications.



Source: Red Hat

Some services provided by K8S

- Find a computer in a cluster with sufficient available capacity to run a container for an application.
- Route network connections between applications to the right container no matter in which computer it is running.
- Run more or less containers of the same application according to demand (scaling).
- Load balance network connections among multiple containers of the same application.
- Start a new container to replace a failed container (or all containers from a failed computer).
- Re-attach applications to its data when their containers restart in different computers.

Source: Red Hat

Things podman/docker doesn't offer or do...

- Solve port mapping hell
- Easy mechanisms for monitoring running containers
- Handling of dead containers
- Methods to distribuite container-based workloads
- Methods to autoscale container instances to handle load

Container orchestration

- A container orchestration platform will perform (among others) the following tasks:
  - Handle the deployment of applications composed by multiple containers (microservices)
  - Scaling the application
    - Only certain components (micro-services) might need to be scaled but consistency in the interactions between containers must be maintained
  - Handle networking
    - Service discovery (allowing containers to find one another)
    - Load balancing

Kubernetes – Architectural elements

- Node(s)
  - Master , Worker
- Cluster
  - Pool of resources (from nodes)
  - Workloads are intelligently distributed across the nodes in the cluster
- Persistent volumes
- Containers
- Pods
  - Grouping of containers (with a common purpose)
  - Pods are the unit of replication in K8S
- Deployments
- Ingress/Load Balancer

Kubernetes and YAML

- Configuration files in Kubernetes are written in YAML !!
  - Pod definition
  - Deployments
  - Services
  - Etc… etc…
- https://www.mirantis.com/blog/introduction-to-yaml-creating-a-kubernetes-deployment/

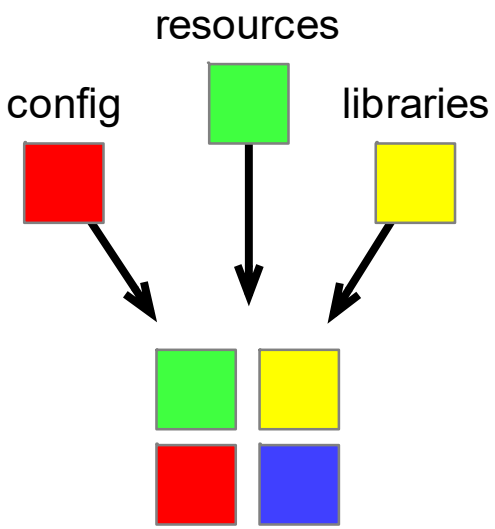
- Putting things together: https://www.youtube.com/watch?v=aSrqRSk43IY

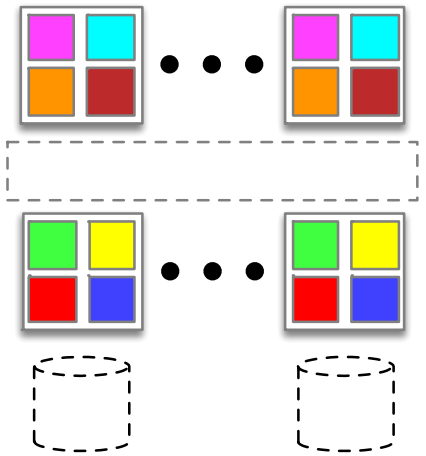
From application build to deployment

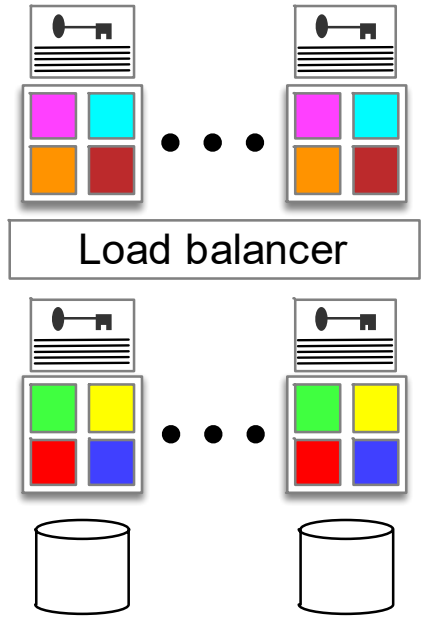
Package

Construct

Deploy







docker/podman build

helm package

helm install/scale

Helm

- Helm is a package manager for Kubernetes
  - Its packages are called “Charts”
- Helm facilitates the deployment of applications composed of many containers (microservices) on a Kubernetes cluster (or clusters)
- Helm charts contain K8S object definitions (in YAML), but add the capacity for customizing them when the chart is installed
- Helm charts are available in several repositories
  - An organization can operate its own chart repository too
- https://www.youtube.com/watch?v=fy8SHvNZGeE

Serverless computing

Serverless computing

- In serverless computing the scaling and the underlying infrastructure for an application is managed by the cloud provider
  - The application automatically drives the allocation and deallocation of resources
  - You do not need to manage the underlying infrastructure at all
- Serverless often adds an event-driven programming model that developers should use
  - Impacts developer's workflow
- From an economic perspective, you pay only per execution (CPU time consumed)

Serverless computing (2)

Variations

- Function as a service (FaaS)
  - AWS Lambda
  - Azure Functions
  - Google Cloud Functions
- Container as a Service (CaaS)
  - Deploy containerized applications without needing to know the underlying infrastructure

Serverless: Any service that lets you consume functionality without having to manage the underlying infrastructure and that uses a pay-only for what you use model

Serverless computing (3)

AWS Lambda

- https://www.youtube.com/watch?v=eOBq__h4OJ4
- Pricing:
  - https://aws.amazon.com/lambda/pricing/

Azure functions:

- https://learn.microsoft.com/en-us/azure/azure-functions/functions-overview
- Pricing:
  - https://azure.microsoft.com/en-us/pricing/details/functions

FaaS vs Containerized service

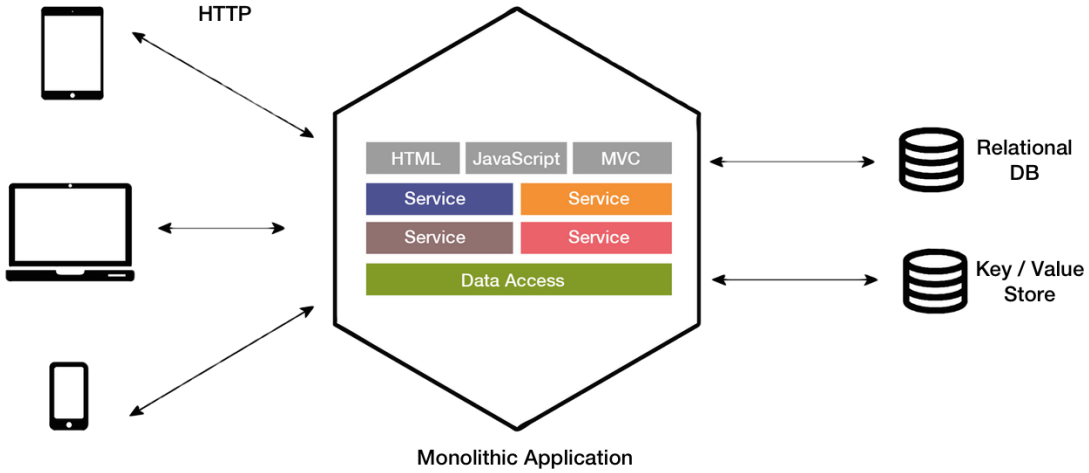
FaaS	Containerized service
Does one thing	Does more than one thing
Can't deploy dependencies	Can deploy dependencies
Must respond to one kind of event	Can respond to more than one kind of event

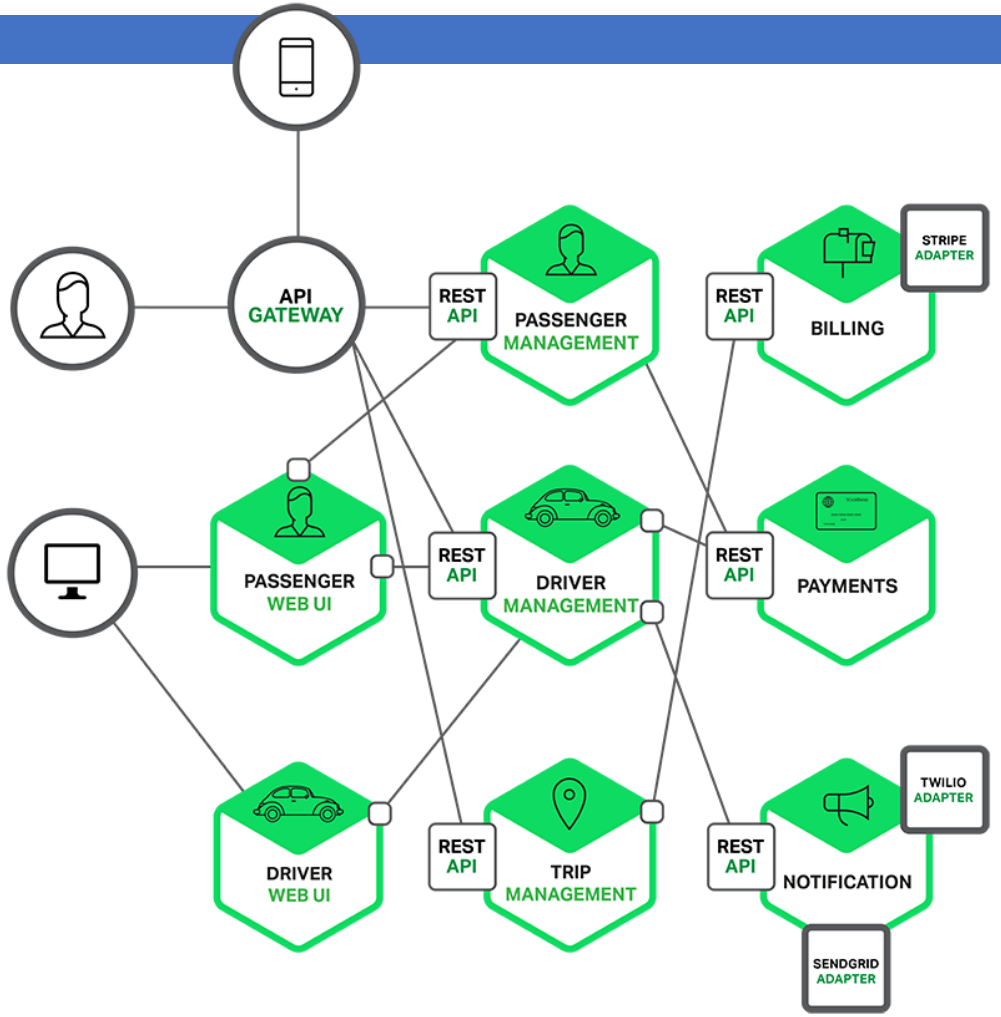
Microservices

Microservices

- The microservice architecture is “an approach to developing a single application as a suite of small services, each running in its own process and communicating with lightweight mechanisms, often an HTTP resource API”
  - https://martinfowler.com/articles/microservices.html
  - https://www.youtube.com/watch?v=CdBtNQZH8a4

Microservices





The path to "Cloud Native" applications

- Brownfield environments: Enterprises that have existing applications that they want to move to the cloud
- Greenfield environment: Enterprises that have new applications and they will start in the cloud from scratch

- Lift and shift
  - Installing software directly into machines (VMs) in the cloud
    - Using IaaS, the enterprise has the most control over the setup of the software.
      - Control also creates responsibilities: managing dependencies, resources, patches, etc.
    - PaaS: Developers don't worry about infrastructure but direct access to the VM environment is typically not allowed
      - PaaS environments based on VMs limit scalability

From Monolith to Micro-Services

- Moving away from monolith applications. Top reasons:
  - Faster deployment
  - Certain parts of the application need to be updated more frequently than others
  - Certain components need different scaling approaches
  - Certain components should be developed differently (i.e: Java vs. C++ vs. Python vs. JavaScript vs….)
  - The code for the application has become too big and complex

From Monolith to Micro-Services (2)

Approaches:

- Strangler pattern: New services or existing components are implemented as microservices
  - As time goes by, more features are operating under the new architecture and the monolithic applications has been transformed into a microservices application
  - https://docs.microsoft.com/en-us/azure/architecture/patterns/strangler-fig
- Anticorruption layer pattern:
  - A software layer manages the communication between the legacy application and new service components
  - Once the monolithic application has become fully cloud native, the anticorruption layer can be removed
  - https://docs.microsoft.com/en-us/azure/architecture/patterns/anti-corruption-layer'
- And many others ….
  - https://learn.microsoft.com/en-us/azure/architecture/patterns/

Running a containerized (microservices) application in the Cloud

- Kubernetes = the new cloud OS
- Most cloud service providers offer “managed Kubernetes” services
  - Setup and runtime of the K8S service is managed
  - Google Kubernetes Engine (GKE)
  - Amazon Elastic Kubernetes Service (EKS)
  - Azure Kubernetes Service (AKS)
- Hybrid and On-Premise
  - Red Hat OpenShift (IBM/RedHat)
  - VMware Tanzu
  - Rancher (SUSE)

From Monolith to Micro-Services (3)

- Optimization of the application
  - Use serverless computing for some containerized workloads
    - Short-lived computations
      - Sending e-mails, sending notifications, updating records
  - Optimize code
  - Optimize cost

Microservices - Benefits

- Agility
  - Each microservice can be built, deployed and tested independently of other components
  - The code base (for each microservice) is more manageable
    - Facilitates the work of the developers (in a team)
  - New features can be added quickly
    - Changes to a microservice can be deployed quickly and rolled back if necessary
- Scalability
  - Each microservice can be scaled independently of other microservice components
- Resilience
  - Failures are more contained and recovery is fast

Microservices - Challenges

- Communication between the microservices introduces latency
  - Network calls are not immediate
  - Network failures and capacity are a concern
- Data integrity and consistency
  - One service might have a relationship (dependency) on data from another service
    - Consistency must be managed/verified
- Versioning
  - Managing forward and backward compatibility between service versions
- Availability
  - If one service depends on another, failures in a service start having a wider impact
    - Manage service dependencies

WebAssembly (WASM)

- WebAssembly is a universal bytecode technology that allows programs written in various languages to be compiled into bytecode, which can be executed directly within web browsers and servers.
  - Languages supported (so far): Go, Rust, and C/C++
- *WebAssembly System Interface (WASI)*
  - an API specification that defines a standard interface between WebAssembly modules and their host environments.
    - Created/announced in March 2019 by the Mozilla foundation
  - WASI allows Wasm modules to access system resources securely, including the network, filesystem, etc.
    - This extremely expanded WebAssembly's potential by enabling it to work not only in browsers but also on servers.

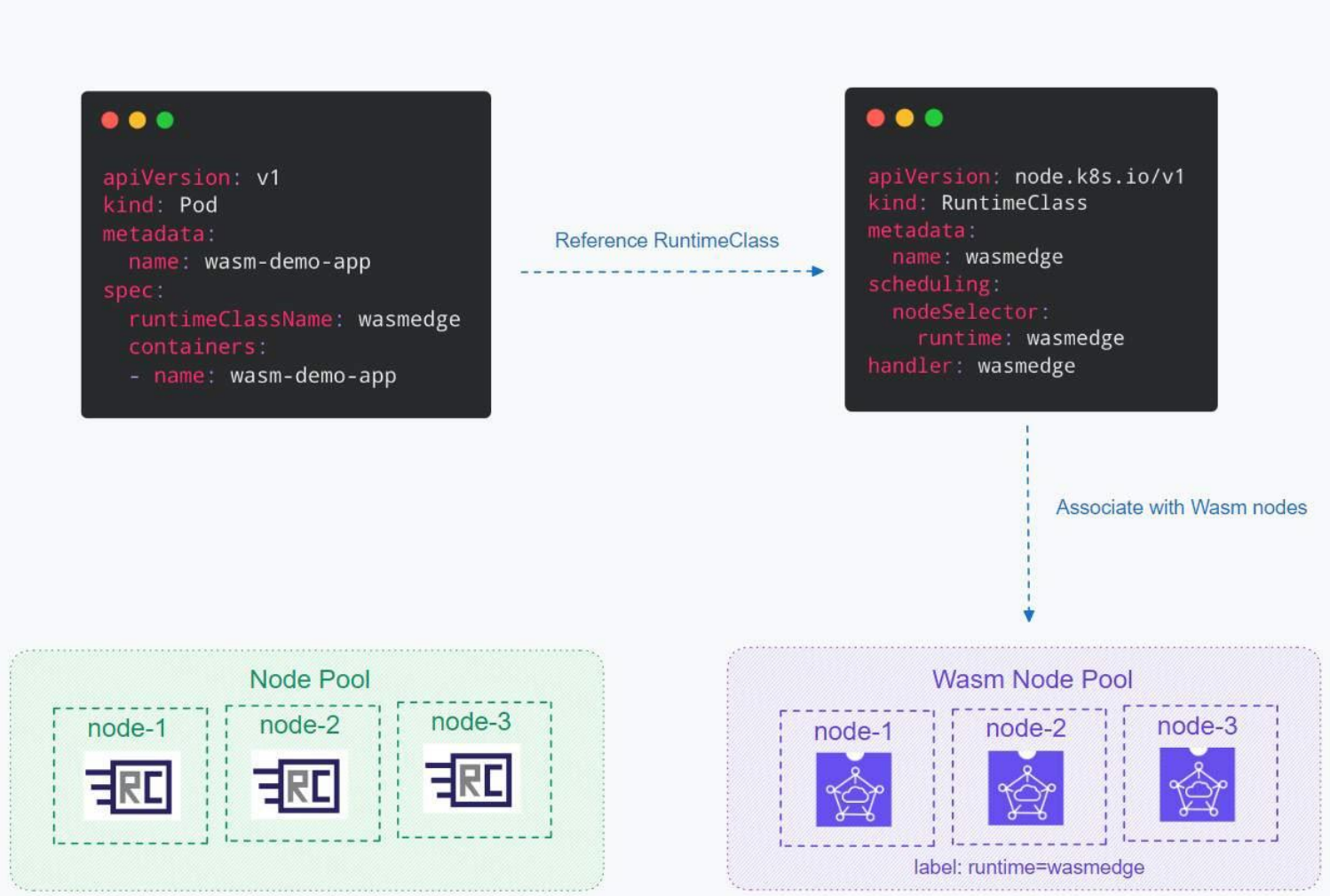
WebAssembly vs. Containers

- **Fast**: Wasm modules typically start within milliseconds, significantly faster than traditional containers, which is crucial for workloads requiring rapid startup, such as serverless functions.
- **Lightweight**: Wasm modules generally occupy less space and demand fewer CPU and memory resources than equivalent container images
- **Secure**: Wasm modules run in a strict sandbox environment, isolated from the underlying host operating system, reducing potential security vulnerabilities.
- **Portable**: Wasm modules can run seamlessly across various platforms and CPU architectures, eliminating the need to maintain multiple container images tailored for different OS and CPU combinations.

WASM and Kubernetes

- To run Wasm workloads on Kubernetes, two key components are needed:
- Worker nodes bootstrapped with a Wasm runtime.
- RuntimeClass objects mapping to nodes with a WebAssembly runtime
  - This addresses the problem of having multiple container runtimes in a Kubernetes cluster, where some nodes might support a Wasm runtime while others support regular container runtimes.
  - You can use RuntimeClass to schedule Wasm workloads specifically to nodes with Wasm runtimes.

WASM and Kubernetes (2)



Source: https://www.cncf.io/blog/2024/03/12/webassembly-on-kubernetes-from-containers-to-wasm-part-01/

Lab 6

- You will need to use the Postman tool or curl for this lab
  - Need to register for a free account to use postman
- Explain curl